Regional Dialect Synthesis Pipeline

Hindi → Haryanvi (Bangru) → Speech

Milestone 6: Deployment & Documentation

Problem Statement

- Haryanvi (Bangru) is a low-resource dialect with limited AI support
- Existing NLP/TTS systems normalize to standard Hindi
- This removes dialect-specific vocabulary, grammar, and prosody
- Creates accessibility issues for rural and native speakers

Why This Problem is Hard

- No large-scale parallel datasets for dialect translation
- Significant lexical + morpho-syntactic divergence from Hindi
- TTS systems fail to capture dialect-specific prosody
- Standard models require large data → not feasible here

Proposed Solution

- LLM-based dialect-aware translation pipeline
- Neural TTS system for speech synthesis
- End-to-end integration using FastAPI
- Designed for real-time inference and deployment

End-to-End Pipeline

- Input: Hindi text
- Step 1: Translation → Haryanvi (Gemma)
- Step 2: Speech synthesis → VITS model
- Output: Natural Haryanvi speech waveform

Dataset Summary

- Text Dataset: 5,594 Hindi–Haryanvi pairs
- Audio Dataset: 5,515 clips
- Filtered Dataset: ~2,764 usable samples (~1.5 hours)
- Includes synthetic + curated dialect data

Training Setup

- Translation: LLaMA 3.1 fine-tuned using QLoRA
- Objective: Learn dialect transformations (not just translation)
- TTS: VITS with transfer learning
- Initialized using Hindi TTS checkpoints

Training vs Deployment (Key Engineering Decision)

Training Phase

- LLaMA 3.1 + QLoRA
- High-quality dialect modeling
- Captures grammar + lexical shifts
- ❌ Slow inference
- ❌ High memory usage

Deployment Phase

- Gemma GGUF model
- Prompt engineering + few-shot
- Faster inference
- Low memory footprint
- Production-ready system

Final System Architecture

- Translation: Gemma GGUF (local inference)
- TTS: Coqui VITS (trained checkpoint)
- Backend: FastAPI
- Frontend: Static HTML UI
- Deployment: GCP VM

Deployment Details

- FastAPI server with async endpoints
- Background model loading at startup
- Thread-based execution for heavy tasks
- Supports JSON + audio output

API Design

- /health → system status
- /api/translate → Hindi → Haryanvi
- /api/tts → Text → speech
- /api/pipeline → full pipeline
- /api/pipeline/base64 → JSON audio output

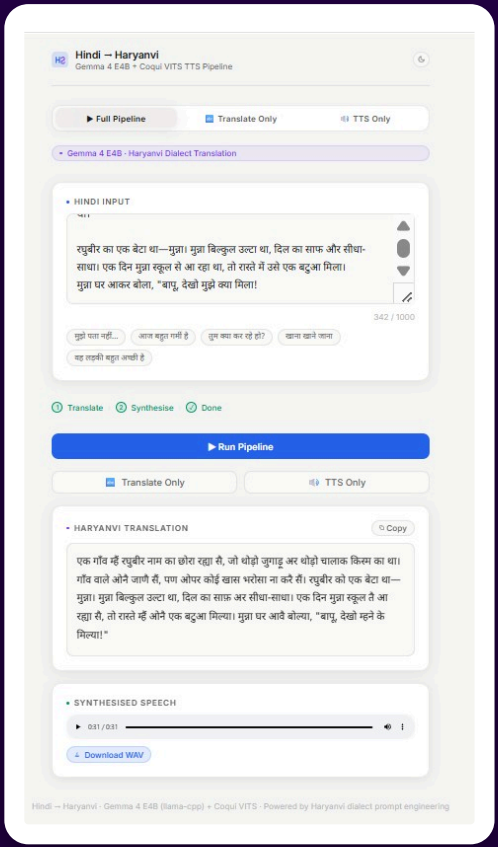
Live API Demo

- Base URL: http://34.123.144.143:8080
- Swagger UI: /docs
- Test Endpoint: /api/pipeline

Example:

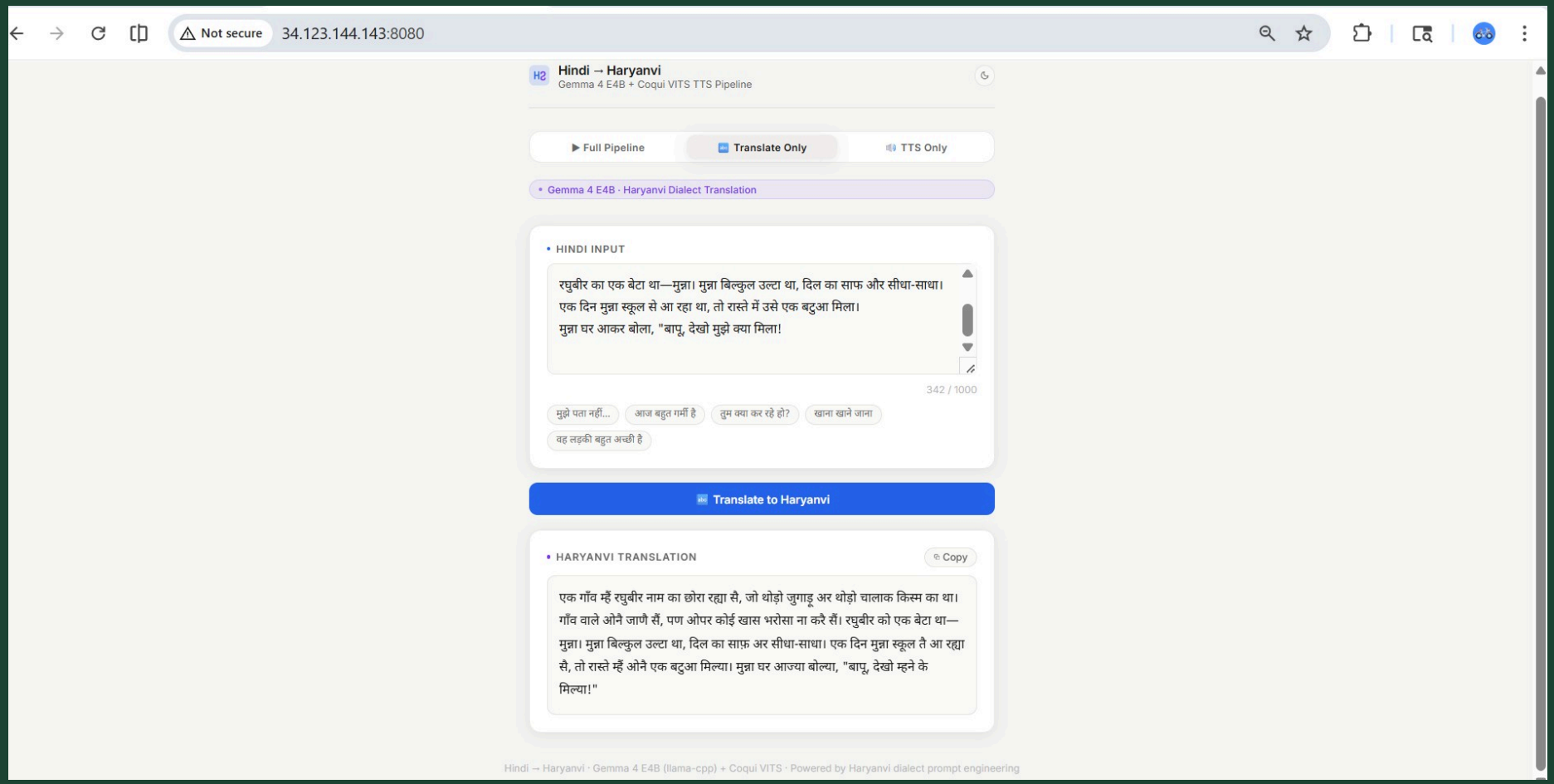
POST /api/pipeline

Demo: Full Pipeline



End-to-end system: Input → Translation → TTS → Audio output

Demo: Translation Output



Performance Considerations

- LLaMA → slow (seconds per request)
- Gemma → significantly faster
- Async API prevents blocking
- Background loading improves startup time

Results

- Dialect-aware translation achieved
- Speech waveform generated successfully
- End-to-end pipeline works in real-time

Limitations

- Residual Hindi influence in translation
- TTS prosody not fully natural
- Limited dataset size

Future Work

- Expand dataset size
- Fine-tune Gemma model
- Improve speech naturalness
- Deploy on Hugging Face / edge devices

Thank You

Questions?